

Website & App Load Speed — the perception budget, and how the AI keeps it there

How the platform measures real load speed, drives it below the threshold at which the human brain can perceive a delay, and keeps it there automatically — on the same AI-optimization convention as everything else, feeding both the existing AI and your custom model. Prepared 2026-09-12.

1. The number (researched, not guessed)

The human brain perceives a response as **instantaneous at about 100 milliseconds** — the classic 0.1-second limit from human-factors research (Jakob Nielsen / Card, Robertson & Mackinlay). Below ~100 ms, a change feels like a direct, instant result of your action; above it, you start to notice a lag.

Per the owner directive, the budget is set **20% below that threshold: 80 ms**, and the optimizer's goal is **"minimize" (no floor)** — so it drives the real numbers **as low as it can**, toward the visual-system floor of roughly one screen frame (~13–16 ms). In other words: the target isn't "fast," it's "below what a person can perceive, then lower."

`PERF_BUDGET_INSTANT_MS = 80` (and the matching route/interaction budgets). Change it in settings if you ever want a different bar.

2. Honest split: what "instant" can and can't cover

A **cold first visit** over a network has a physical floor no code can remove — DNS lookup, TLS handshake, and at least one round-trip to the server all have to happen before a single byte of the page arrives, which is tens of milliseconds at best and usually more. So the 80 ms wall is applied to **what is actually controllable**:

- **In-app navigation** (clicking around once the app is open) — target < 80 ms.
- **Interaction latency** (does a tap/click feel instant) — target < 80 ms.
- **Warm / repeat loads** (the service worker already has the app) — target < 80 ms.

Cold largest-contentful-paint keeps a separate, network-realistic budget (`PERF_BUDGET_LCP_MS = 1000`), and the code still pushes it as low as possible (preconnect, small critical shell, edge/CDN caching).

3. What's built to get there

Measurement (from day one). `src/lib/perf-vitals.js` captures real load speed from every visitor using only native browser APIs — **zero dependencies, never blocks rendering** — and beacons it to `perfVitalsIngest`: largest-contentful-paint, interaction-to-next-paint, first paint, time-to-first-byte, layout stability, and **in-app navigation time** (click → the new page is shown). No PII: only the metric, the milliseconds, and the route name. It samples per `PERF_SAMPLE_RATE` (default: everyone) and flushes on page-hide so nothing is lost.

Route prefetch. `src/lib/route-prefetch.js` warms the *next* page's code before you click it — on hover, on focus, or as the link scrolls into view — so navigation commits in a few milliseconds. Aggressiveness is AI-tuned (`PERF_PREFETCH_LEVEL`: off / hover / visible / eager) and it backs off on save-data / slow connections.

Preconnect. The app opens the DNS + TLS connection to the backend API origin at startup, so the first data call doesn't pay for the handshake.

Already in place and reinforced. A PWA service worker precaches the whole app shell (so repeat loads are warm), code-splitting keeps heavy pages out of the initial download, and the React Query cache serves already-loaded data instantly (freshness window is `PERF_QUERY_STALE_MINUTES`, tunable without a redeploy).

4. The AI keeps it fast — forever, and reverts regressions

Speed isn't a one-time fix; it's a metric the AI **watches and defends continuously**, on the *same* convention as every other optimization:

- Load-time is registered as an **optimizable knob** in the existing optimizer: `PERF_PREFETCH_LEVEL` with the objective `route_nav_p75_ms` and goal `min`. The optimizer collects the p75 of real navigation times, proposes a change, applies it within bounds, and — crucially — **reverts it on the next cycle if it didn't actually help** (the standard revert-on-regression the engine already uses for engagement knobs).
- `perfOptimize` runs this every 6 hours (scheduled). Auto-optimization is **on from day one** (`PERF_AUTO_OPTIMIZE_ENABLED = 1`).
- The **build-time guard** (`deploy-kit/audit.mjs`, structural check 8) fails the build if the entry bundle's gzipped size ever crosses the load-speed budget — so a future change can't silently bloat the app and push load times back up. It catches the most common way speed regresses: a heavy new dependency landing in the shared chunk.

So there are two independent safety nets: the runtime optimizer (reverts a live change that hurt speed) and the build guard (blocks a code change that would hurt speed before it ships).

4a. Preload the whole app while the user waits

The moment a load crosses the load-time threshold and the earn-while-loading questions appear (see `EARN-WHILE-LOADING-SURVEY.md`), the app has the user's attention for a few seconds — so it uses that time to **preload the entire app's page code in the background**. By the time the current load finishes and they start navigating, every screen is already warm and commits instantly.

This is deliberately **budget-aware** — it tests as it goes and stays within the 80 ms budget so it never makes the foreground feel slow while it works:

- It **skips entirely on save-data / very slow connections** (never competes for scarce bandwidth).
- It warms **one page chunk at a time, strictly during browser idle time** — never a burst that floods the network or blocks the main thread.
- It **times each warm and backs off**: if a chunk takes too long (a sign the device or link is congested), it widens the gap between warms; when things are quick again, it tightens back up. So main-thread work stays in small slices well under the 80 ms interaction budget.
- It runs at most once per session.

Toggle: `PERF_PRELOAD_ON_WAIT_ENABLED` (default on).

5. It runs through BOTH AI models (existing + custom), by convention

Load-time optimization is **not a separate silo** — it's wired into the same decision stream as the rest of the platform's AI, so both models cover it:

- **The existing AI** optimizes it directly: `perfOptimize` calls the normal optimizer pass, so the pre-existing optimization engine is the one tuning load speed.
- **A new autonomy domain**, `load_time` (`auto_ok`, reversible, non-sensitive — never money / identity / legal), is added to the kernel. Every load-speed change `perfOptimize` makes is recorded as an `AutonomyDecision` in that domain, exactly like every other AI decision.
- Because it's now a normal domain in the decision stream, it **automatically feeds your custom model's training data** (`model-training.ts` reads these decisions/outcomes) **and appears as its own "function"** in the function-by-function switch gate (`model-eval.ts`). Your custom model must beat the existing AI on the `load_time` function too — individually and as a whole — before it can switch over. Nothing extra had to be wired for that; it falls out of using the shared convention.

The result: load-time optimization fits the AI-model conventions you already built — the existing model optimizes it, and the custom model learns it and is graded on it, on the same rules as pricing, engagement, moderation, and every other function.

6. Where you see it

The **Data-Driven Coverage** dashboard has a **Load speed** card: each vital's p75 vs its budget (green = under 80% of budget = feels instant; amber = within budget; red = over → the optimizer acts), whether auto-optimization is on, the active prefetch strategy, and the in-app-navigation trend over time.

7. Settings (all in Automation; on from day one)

`PERF_MONITORING_ENABLED` (capture, default on), `PERF_AUTO_OPTIMIZE_ENABLED` (AI tuning, default on), `PERF_PREFETCH_ENABLED` + `PERF_PREFETCH_LEVEL` (0-3, AI-tuned), `PERF_QUERY_STALE_MINUTES` (client freshness), `PERF_SAMPLE_RATE`, and the budgets `PERF_BUDGET_INSTANT_MS = 80`, `PERF_BUDGET_ROUTE_MS = 80`, `PERF_BUDGET_INP_MS = 80`, `PERF_BUDGET_LCP_MS = 1000` (cold, network-bound).

8. Honest limits

- **Cold first paint is network-bound.** No client code removes DNS/TLS/round-trip latency; that's what CDN/edge hosting and the preconnect address, and why cold LCP has its own realistic budget separate from the 80 ms wall.
- **The perception budget is applied to controllable metrics** (navigation, interaction, warm loads). Those are the ones a user actually experiences as "the app," and those are the ones held below the ~100 ms threshold.
- **Telemetry is real-user (p75).** The numbers on the dashboard come from actual visitors, so they populate as traffic arrives; before launch the card shows the budgets with "no data yet."

9. Where it lives (for the reviewer / developer)

- Client: `src/lib/perf-vitals.js` (capture + beacon), `src/lib/route-prefetch.js` (prefetch), `src/main.jsx` (init + preconnect), `src/lib/NavigationTracker.jsx` (route-timing hook).
- SDK: `backend/sdk/perf-optimizer.ts` (budget math + p75 + getters), `backend/sdk/optimizer.ts` (the `PERF_PREFETCH_LEVEL` knob + perf metrics in `collectSignals`), `backend/sdk/autonomy-kernel.ts` (`load_time` domain).
- Functions: `perfVitalsIngest` (public beacon), `perfConfig` (public client knobs), `perfStatus` (admin read), `perfOptimize` (scheduled, every 6h).
- Guard: `deploy-kit/audit.mjs` structural check 8 (entry-bundle gzip budget).
- Dashboard: **Data-Driven Coverage** → the **Load speed** card.